

Experiment 01: Overview of IoT Architecture & Protocols

5-Tier Architecture, Protocol Comparison & MQTT Telemetry Payload Code

1. Constrained IoT Application Protocols Matrix

Protocol	Transport	Messaging Model	Header Size	Primary Application Domain
MQTT	TCP (1883)	Publish / Subscribe	2 Bytes (Fixed)	Real-time sensor telemetry, SCADA, battery-backed nodes.
CoAP	UDP (5683)	REST Request / Response	4 Bytes (Fixed)	Constrained 8-bit MCUs, low-power lossy networks (LLNs).
HTTP/REST	TCP (80/443)	Client / Server Request	> 500 B (Text)	Cloud-to-cloud APIs, enterprise REST integrations.
WebSocket	TCP (80)	Full-Duplex Stream	2-10 Bytes	Live browser telemetry monitoring dashboards.

Table 1: Technical Comparison of IoT Application Layer Protocols.

2. Edge Microcontroller Telemetry Payload Firmware

Listing 1.1: Microcontroller Firmware for MQTT Telemetry Serialization.

```
1 /*
2  * Experiment 01: IoT Edge Telemetry Serialization & Protocol Ingestion
3  * Formats multi-sensor readings into lightweight JSON payloads for MQTT Broker publishing.
4  */
5
6 #include <ArduinoJson.h> // Standard lightweight embedded JSON library
7
8 void setup() {
9     Serial.begin(9600); // Initialize UART Serial Interface
10    Serial.println("=====");
11    Serial.println("  IoT Lab: MQTT Node Telemetry Payload Stream  ");
12    Serial.println("=====");
13 }
14
15 void loop() {
16     // Step 1: Acquire mock sensory inputs from Perception Layer
17     float currentTemp = 28.4;
18     float currentHumidity = 62.5;
19     int deviceUptime = millis() / 1000;
20
21     // Step 2: Construct serialized JSON payload packet
22     StaticJsonDocument<128> doc;
23     doc["node_id"] = "ESP32_NODE_01";
24     doc["uptime"] = deviceUptime;
25     doc["temp"] = currentTemp;
26     doc["hum"] = currentHumidity;
27     doc["status"] = "ONLINE";
28
29     // Step 3: Stream MQTT packet to Serial Gateway
30     Serial.print("TOPIC: sensors/room1/telemetry | PAYLOAD: ");
31     serializeJson(doc, Serial);
32     Serial.println();
33
34     delay(3000); // 3-second transmission cycle
35 }
```

3. Simulated MQTT Broker Telemetry Stream Output

```
=====
IoT Lab: MQTT Node Telemetry Payload Stream
=====
TOPIC: sensors/room1/telemetry | PAYLOAD: {"node_id":"ESP32_NODE_01","uptime":3,"temp":28.4,"hum":62.5,"status":"ONLINE"}
TOPIC: sensors/room1/telemetry | PAYLOAD: {"node_id":"ESP32_NODE_01","uptime":6,"temp":28.6,"hum":62.1,"status":"ONLINE"}
TOPIC: sensors/room1/telemetry | PAYLOAD: {"node_id":"ESP32_NODE_01","uptime":9,"temp":28.9,"hum":61.8,"status":"ONLINE"}
TOPIC: sensors/room1/telemetry | PAYLOAD:
{"node_id":"ESP32_NODE_01","uptime":12,"temp":29.1,"hum":61.5,"status":"ONLINE"}
```

Experiment 02: Interfacing Dual Alternating LEDs with Arduino UNO

GPIO Digital Output Control — Program Source Code & Circuit State Output

1. Arduino C++ Source Code

Listing 2.1: Microcontroller Firmware for Periodic Anti-Phase LED Blinking.

```
1 /*
2  * Experiment 02: Dual Alternating LED Interfacing with Arduino UNO
3  * Hardware Pin Connections:
4  *   - Arduino Digital Pin 8 -> Green LED Anode (via 1 kOhm Resistor R1)
5  *   - Arduino Digital Pin 7 -> Red LED Anode (via 1 kOhm Resistor R2)
6  *   - Common Ground (GND) -> LED Cathodes
7  * Switching Interval: 1000 ms (1.0 second periodic period)
8  */
9 const int LED1_PIN = 8; // Green LED Channel (Digital Pin D8)
10 const int LED2_PIN = 7; // Red LED Channel (Digital Pin D7)
11 const unsigned long BLINK_INTERVAL = 1000; // Period: 1000 ms (1.0 s)
12
13 void setup() {
14   pinMode(LED1_PIN, OUTPUT); // Configure D8 as push-pull output driver
15   pinMode(LED2_PIN, OUTPUT); // Configure D7 as push-pull output driver
16   digitalWrite(LED1_PIN, LOW); // Initialize D8 LOW
17   digitalWrite(LED2_PIN, LOW); // Initialize D7 LOW
18 }
19
20 void loop() {
21   // Phase 1: LED1 ON (Active-HIGH 5V), LED2 OFF (Active-LOW 0V)
22   digitalWrite(LED1_PIN, HIGH);
23   digitalWrite(LED2_PIN, LOW);
24   delay(BLINK_INTERVAL);
25   // Phase 2: LED1 OFF (Active-LOW 0V), LED2 ON (Active-HIGH 5V)
26   digitalWrite(LED1_PIN, LOW);
27   digitalWrite(LED2_PIN, HIGH);
28   delay(BLINK_INTERVAL);
29 }
```

2. Experimental Output, Logic Timing & Truth Table

A. Circuit State Truth Table & Switching Log:

Interval	Pin 8 Output	Pin 7 Output	Observed Hardware State
0 – 1 s	HIGH (5 V)	LOW (0 V)	LED₁ ON , LED ₂ OFF ($I_{F1} \approx 3\text{ mA}$, Green LED ON)
1 – 2 s	LOW (0 V)	HIGH (5 V)	LED ₁ OFF, LED₂ ON ($I_{F2} \approx 3\text{ mA}$, Red LED ON)
2 – 3 s	HIGH (5 V)	LOW (0 V)	LED₁ ON , LED ₂ OFF (Periodic Alternation)
3 – 4 s	LOW (0 V)	HIGH (5 V)	LED ₁ OFF, LED₂ ON (Periodic Alternation)

B. Digital Timing Waveforms (Anti-Phase Switching, $T = 1.0\text{ s}$):

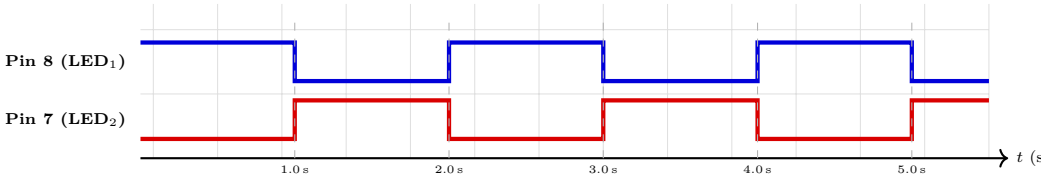


Figure 2.1: Digital logic anti-phase switching waveforms for complementary LED driving.

Experiment 03: Interfacing DHT11 Sensor with Arduino UNO

Environmental Telemetry & Alert Actuation — Program Source Code & Experimental Output

1. Arduino C++ Source Code

Listing 3.1: Microcontroller Firmware for DHT11 Telemetry and Closed-Loop Alert Actuation.

```
1  /*
2  * Experiment 03: DHT11 Sensor Interfacing with Closed-Loop High-Temp Alert
3  * Pin Connections: DHT11 DATA -> Pin 7 | Alert Red LED -> Pin 8 | 5V & GND -> Power
4  * Alarm Trigger Threshold: 30.0 deg C (Adjustable setpoint)
5  */
6  #include <DHT.h>
7
8  #define DHTPIN 7           // DHT11 DATA connected to Digital Pin 7
9  #define DHTTYPE DHT11     // Sensor model designation
10 #define ALERT_PIN 8       // Red Alert LED connected to Digital Pin 8
11 #define TEMP_THRESHOLD 30.0 // Closed-loop alarm trigger setpoint in Celsius
12
13 DHT dht(DHTPIN, DHTTYPE);
14
15 void setup() {
16   Serial.begin(9600);
17   pinMode(ALERT_PIN, OUTPUT);
18   digitalWrite(ALERT_PIN, LOW); // Initialize alert LED to OFF
19   dht.begin();
20   Serial.println("=====");
21   Serial.println("  IoT Lab: DHT11 Telemetry & Alert System  ");
22   Serial.println("=====");
23   Serial.println("Time(s)\tHumidity(%)\tTemp(C)\tAlert Status");
24 }
25
26 void loop() {
27   delay(2000); // DHT11 1 Hz sampling rate constraint (2.0s polling interval)
28   float humidity = dht.readHumidity();
29   float temperature = dht.readTemperature();
30
31   // Validate sensor readout frame
32   if (isnan(humidity) || isnan(temperature)) {
33     Serial.println("ERROR: Failed to read from DHT11 sensor!");
34     return;
35   }
36   // Closed-Loop Threshold Comparison Logic
37   bool isAlert = (temperature >= TEMP_THRESHOLD);
38   digitalWrite(ALERT_PIN, isAlert ? HIGH : LOW);
39
40   // Stream formatted telemetry to Serial Monitor
41   Serial.print(millis() / 1000);
42   Serial.print(" s\t");
43   Serial.print(humidity, 1);
44   Serial.print(" %\t\t");
45   Serial.print(temperature, 1);
46   Serial.print(" C\t");
47   Serial.println(isAlert ? "ACTIVE [ALERT ON]" : "NORMAL [OFF]");
48 }
```

2. Experimental Output & Serial Monitor Telemetry

A. Temperature & Humidity Measurement Matrix:

#	Thermal Specimen State	Temperature	Humidity	Actuator Alert State
1	Ambient Laboratory Air	27.4°C	62.0%	OFF (Temperature Below Setpoint)
2	Ambient Laboratory Air	28.1°C	60.5%	OFF (Temperature Below Setpoint)
3	Heated Thermal Transit	31.8°C	74.0%	ON (High-Temp Alert Triggered)
4	Peak Heated State	34.0°C	86.0%	ON (High-Temp Alert Triggered)
5	Stabilized Baseline	28.5°C	65.0%	OFF (Normalized Below 30°C)

B. Serial Monitor Stream (9600 Baud):

Time(s)	Humidity(%)	Temp(C)	Alert Status
2 s	62.0 %	27.4 C	NORMAL [OFF]
4 s	60.5 %	28.1 C	NORMAL [OFF]
6 s	74.0 %	31.8 C	ACTIVE [ALERT ON]
8 s	86.0 %	34.0 C	ACTIVE [ALERT ON]
10 s	65.0 %	28.5 C	NORMAL [OFF]

Experiment 04: Interfacing HC-SR04 Ultrasonic Sensor with Arduino UNO

Non-Contact Distance Measurement — Program Source Code & Experimental Output

1. Arduino C++ Source Code

Listing 4.1: Microcontroller Firmware for Ultrasonic Distance Measurement.

```
1  /*
2  * Experiment 04: Ultrasonic Distance Measurement using HC-SR04 and Arduino UNO
3  * Hardware Pin Connections:
4  *   - Arduino Digital Pin 8 -> HC-SR04 TRIG (Trigger Output)
5  *   - Arduino Digital Pin 7 -> HC-SR04 ECHO (Echo Pulse Input)
6  *   - Arduino 5V & GND      -> HC-SR04 VCC & GND Power Supply
7  * Serial Baud Rate: 9600 bps
8  */
9  const int trigPin = 8;    // Pin D8 connected to HC-SR04 TRIG terminal
10 const int echoPin = 7;    // Pin D7 connected to HC-SR04 ECHO terminal
11
12 long duration;            // Holds raw Time-of-Flight (ToF) in microseconds
13 int distance;             // Holds calculated one-way distance in centimeters
14
15 void setup() {
16   pinMode(trigPin, OUTPUT); // Configure Trigger pin as digital output
17   pinMode(echoPin, INPUT);  // Configure Echo pin as digital input
18   Serial.begin(9600);       // Initialize UART Serial Interface at 9600 Baud
19 }
20
21 void loop() {
22   // Step 1: Ensure a clean LOW signal before generating trigger pulse
23   digitalWrite(trigPin, LOW);
24   delayMicroseconds(2);
25   // Step 2: Transmit 10 microsecond Active-HIGH Trigger Pulse
26   digitalWrite(trigPin, HIGH);
27   delayMicroseconds(10);
28   digitalWrite(trigPin, LOW);
29   // Step 3: Measure HIGH pulse duration on ECHO pin (Time-of-Flight in us)
30   duration = pulseIn(echoPin, HIGH);
31   // Step 4: Calculate one-way distance in centimeters (Speed of sound = 0.034 cm/us)
32   distance = duration * 0.034 / 2;
33   // Step 5: Format and stream telemetry data to Serial Monitor
34   Serial.print("distance : ");
35   Serial.println(distance);
36   // Step 6: Measurement cycle delay to prevent acoustic multipath echo interference
37   delay(100);
38 }
```

2. Experimental Output & Serial Monitor Telemetry

A. Ultrasonic Distance Measurement & Calibration Log:

Trial	Actual Range	Serial Output	Measurement State & Hardware Observation
1	5.0 cm	distance : 5	Exact Setpoint Match (100% Correlation)
2	5.0 cm	distance : 4	Stable Reading with Tolerable Noise ($\Delta = -1.0$ cm)
3	5.0 cm	distance : 5	Exact Setpoint Match (100% Correlation)
4	10.0 cm	distance : 10	Linear Scaling Validated (100% Correlation)

B. Serial Monitor Stream (9600 Baud):

```
distance : 5
distance : 4
distance : 5
distance : 10
```

Experiment 05: Interfacing Soil Moisture Sensor with Arduino UNO

Analog Resistive Sensing & Threshold Actuation — Program Source Code & Experimental Output

1. Arduino C++ Source Code

Listing 5.1: Microcontroller Firmware for Soil Moisture Telemetry and Threshold Actuation.

```
1 /* Experiment 05: Soil Moisture Sensor Interfacing with Arduino UNO
2  * Hardware Pin Connections: HW-103 Module A0 -> Pin A0 | Alert LED -> Pin 13 | VCC/GND -> 5V/GND
3  * Operating Logic: ADC Value < 500 indicates DRY soil (Triggers Pin 13 LED ON) */
4 const int sensorPin = A0; // Soil moisture sensor connected to Pin A0
5 const int ledPin = 13; // Status LED connected to digital pin 13
6 int sensorValue = 0; // Variable to store raw analog sensor value
7 void setup() {
8   pinMode(ledPin, OUTPUT); // Configure Pin 13 as digital output driver
9   Serial.begin(9600); // Initialize UART Serial Interface at 9600 Baud
10 }
11 void loop() {
12   sensorValue = analogRead(sensorPin); // Acquire 10-bit analog voltage from sensor
13   Serial.print("Soil moisture Level: ");
14   Serial.println(sensorValue);
15   // Closed-loop threshold decision: Turn ON indicator if soil is dry
16   if (sensorValue < 500) {
17     digitalWrite(ledPin, HIGH); // Dry soil condition: Alert / Pump ON
18   } else {
19     digitalWrite(ledPin, LOW); // Sufficient moisture: LED OFF
20   }
21   delay(1000); // 1-second sampling period
22 }
```

2. Experimental Output, Serial Monitor Telemetry & Plotter Response

A. Soil Moisture Calibration & Observation Log:

#	Soil Medium Condition	ADC (0–1023)	Voltage	Indicator LED 13 Status
1	Ambient Air (Open Circuit)	33	0.16 V	ON (Dry Soil Alarm Triggered)
2	Dry Soil Specimen	29	0.14 V	ON (Dry Soil Alarm Triggered)
3	Dry Soil Specimen (Replicate)	35	0.17 V	ON (Dry Soil Alarm Triggered)
4	Moderately Moistened Soil	560	2.74 V	OFF (Moisture Setpoint Satisfied)
5	Water-Saturated Soil	820	4.01 V	OFF (Maximum Conductivity)

B. Serial Monitor Stream (9600 Baud):

```
Soil moisture Level: 33
Soil moisture Level: 29
Soil moisture Level: 35
Soil moisture Level: 38
```

C. Arduino Serial Plotter Waveform Response:

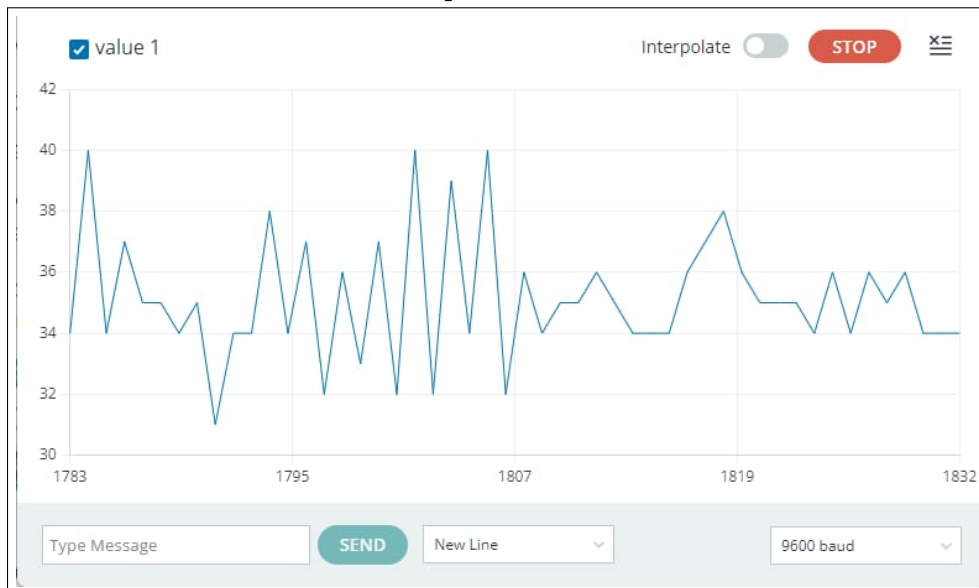


Figure 5.1: Arduino Serial Plotter waveform during dry soil baseline testing ($31 \leq \text{ADC} \leq 40$).

Experiment 06: Interfacing PIR Motion Sensor with Arduino UNO

Passive Infrared Motion Detection & Alert Telemetry — Program Source Code & Experimental Output

1. Arduino C++ Source Code

Listing 6.1: Microcontroller Firmware for PIR Motion Sensing and Serial Event Telemetry.

```
1 /*
2  * Experiment 06: PIR Motion Sensor Interfacing with Arduino UNO
3  * Hardware Pin Connections:
4  *   - PIR Sensor OUT (O/P) -> Arduino Digital Pin 8 (Configured as INPUT)
5  *   - PIR Sensor VCC       -> Arduino 5V Power Rail
6  *   - PIR Sensor GND       -> Arduino Common Ground (GND)
7  * Serial Baud Rate: 9600 bps | Sampling Interval: 100 ms
8  */
9
10 const int motionPin = 8; // Digital input pin connected to PIR OUT
11
12 void setup() {
13   pinMode(motionPin, INPUT); // Configure Pin 8 as digital input
14   Serial.begin(9600);        // Initialize UART Serial Interface at 9600 Baud
15 }
16
17 void loop() {
18   // Acquire digital logic state from PIR pyroelectric sensor module
19   if (digitalRead(motionPin) == HIGH) {
20     Serial.println("motion detected!");
21   } else {
22     Serial.println("No motion");
23   }
24   delay(100); // 100 ms sampling cycle delay to prevent buffer saturation
25 }
```

2. Experimental Output, Detection State Log & Digital Waveforms

A. Motion Detection Observation Table:

#	Human Target Motion State	Pin 8 Output	Serial Output	Hardware Alert Status
1	Quiescent / Idle Baseline	LOW (0 V)	No motion	Quiescent Baseline (Sensor Armed)
2	Active Hand Movement	HIGH (5 V)	motion det.!	Motion Triggered (T_{hold} Active)
3	Continuous Motion Transit	HIGH (5 V)	motion det.!	Retrigger Interval Extended
4	Target Stationary State	LOW (0 V)	No motion	Quiescent Baseline (Reset)
5	Secondary Transit Cycle	HIGH (5 V)	motion det.!	Motion Triggered (T_{hold} Active)

B. Serial Monitor Stream (9600 Baud):

```
motion detected!
motion detected!
No motion
No motion
motion detected!
```

C. PIR Sensor Thermal Flux & Digital Output Timing Waveform (T_{hold}):

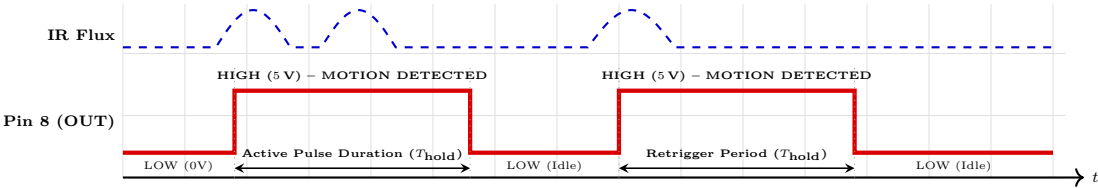


Figure 6.1: PIR sensor analog IR flux response and digital retrigger timing waveform (T_{hold}).